

EXAMEN FINAL ARQUITECTURA DE COMPUTADORAS**10/12/2015**

EJERCICIO 1: (45%) En el intervalo 0x7B a 0x8E se encuentran almacenados números codificados en BCD de dos dígitos. Realice un programa en lenguaje de máquina que sume los contenidos de las posiciones contiguas. Cada número debe sumarse una sola vez. El resultado en BCD de las sumas debe almacenarse a partir de la posición 0x10.

- ☐ Diagrama de Flujo
- ☐ Definición del R.I.
- ☐ Definición del set de instrucciones
- ☐ Programa en lenguaje de máquina

EJERCICIO 2: (10%) Verifique, utilizando el método de HAMMING, si la cadena 1000110001011000 es correcta, en caso de no serlo indique la posición del error. Extraiga los bits de paridad y considerando que los bits restantes contienen información en base 9, conviértalos a base 3 utilizando pasaje directo.

EJERCICIO 3: (45%) Dibuje un registro AC (con F-F 'T'), de 4 bits que posean la lógica combinacional necesaria para poder realizar:

Cod. Op	Operación
000	Sumador Base 13 $AC \leftarrow (AC) + (Bus M)$
001	Poner a uno todos los bits
010	Transferir el complemento a la base desde el AC al Bus M
011	Detección del contenido de AC igual a 1011
100	Desplazamiento cíclico a izquierda del AC

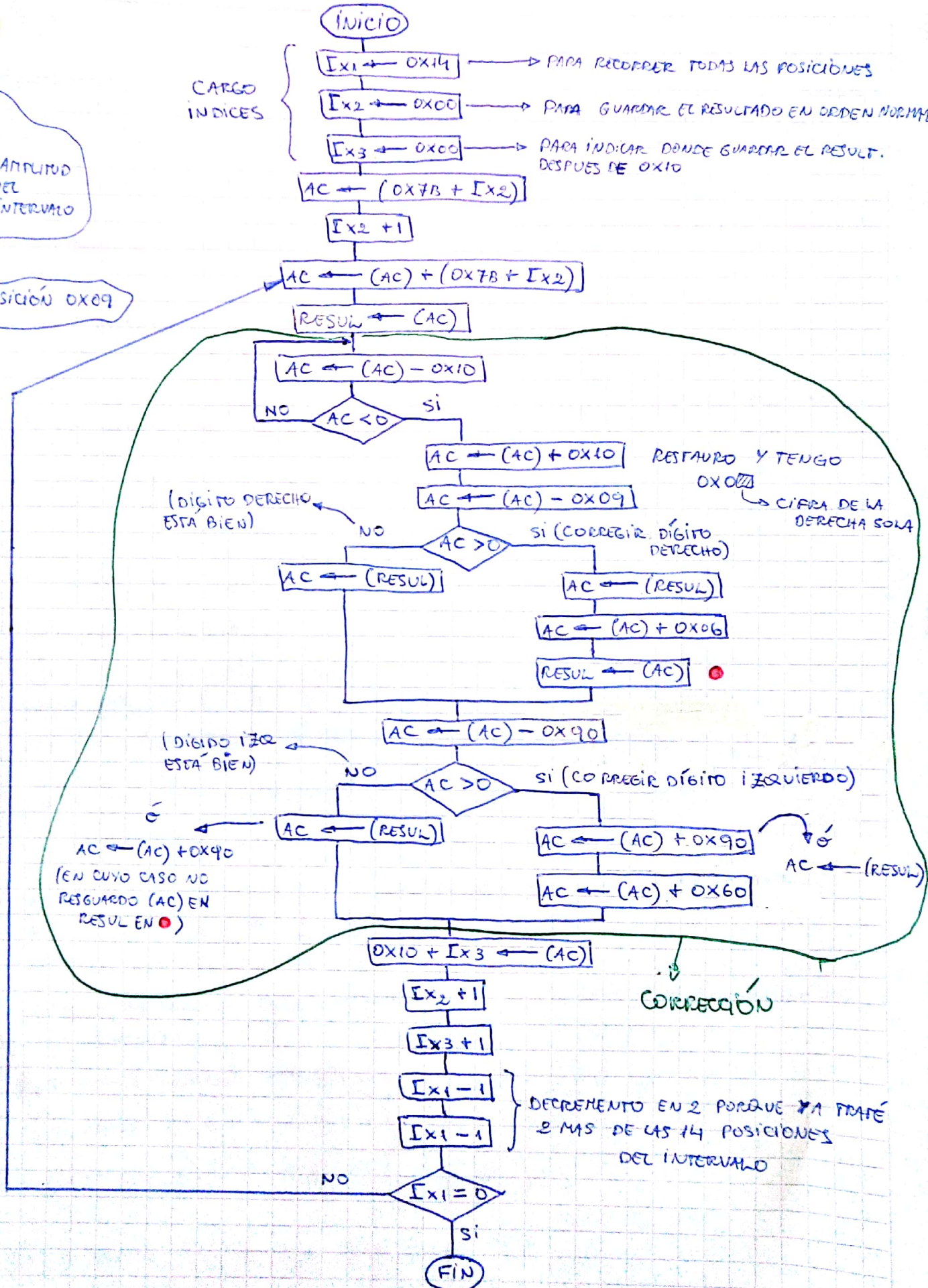
Realizar el diseño del sumador, detector y corrector.

FINAL 10/12/15

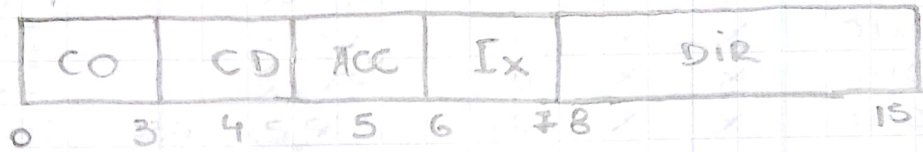
1

BE
- FB
13 → AMPLITUD
DEL
INTERVALO

RESUL = POSICIÓN 0X09



b)



c)

0000 - FIN

0001 - CARGAR INDICE

0010 - CARGAR ACUMULADOR ~~000000~~ 0 ACC

0011 - INCREMENTAR INDICE

0100 - SUMAR AL ACUMULADOR

0101 - ALMACENAR ACUMULADOR ~~000000~~ 0 ACC

0110 - DECREMENTAR INDICE

0111 - SALTO CONDICIONAL SI $I_x = 0$

1000 - RESTAR AL ACUMULADOR

1001 - SALTO CONDICIONAL SI $AC > 0$

1010 - SALTO INCONDICIONAL

CD $\begin{cases} 0 = \text{DIRECTO} \\ 1 = \text{INMEDIATO} \end{cases}$

ACC $\begin{cases} 0 = \text{NO USA} \\ 1 = \text{USA} \end{cases}$

I_x $\begin{cases} 00 = \text{NO USA} \\ 01 = I_{x1} \\ 10 = I_{x2} \\ 11 = I_{x3} \end{cases}$

d)

DIREC. MEMORIA	COD.OP.	CD	ACC	I_x	DIR IMPLICADA	COMENT
0x30	0001	1	x	01	0x14	$I_{x1} \leftarrow 0x14$
0x31	0001	1	x	10	0x00	$I_{x2} \leftarrow 0x00$
0x32	0001	1	x	11	0x00	$I_{x3} \leftarrow 0x00$
0x33	0010	0	0	10	0x78	$AC \leftarrow (0x78 + I_{x2})$
0x34	0011	x	x	10	xxxx	$I_{x2} + 1$
0x35	0100	0	0	10	0x7B	$AC \leftarrow (AC) + (0x7B + I_{x2})$
0x36	1000	1	0	00	0x99	$AC \leftarrow (AC) - 0x99$
0x37	1001	x	0	00	0x3A	SALTO SI $AC > 0$
0x38	0100	1	0	00	0x99	$AC \leftarrow (AC) + 0x99$
0x39	1010	x	x	00	0x3C	SALTO INCONDICIONAL
0x3A	0100	1	0	00	0x99	$AC \leftarrow (AC) + 0x99$
0x3B	0100	1	0	00	0x60	$AC \leftarrow (AC) + 0x60$
0x3C	0010	1	1	00	0x00	$ACC \leftarrow 0x00$
0x3D	1000	1	0	00	0x10	$AC \leftarrow (AC) - 0x10$
0x3E	0100	1	1	00	0x10	$ACC \leftarrow (ACC) + 0x10$
0x3F	1001	x	0	00	0x3D	SALTO SI $AC > 0$
0x40	0100	1	0	00	0x10	$AC \leftarrow (AC) + 0x10$
0x41	1000	1	1	00	0x10	$ACC \leftarrow (ACC) - 0x10$
0x42	1000	1	0	00	0x09	$AC \leftarrow (AC) - 0x09$
0x43	1001	x	0	00	0x46	SALTO SI $AC > 0$
0x44	0100	1	0	00	0x09	$AC \leftarrow (AC) + 0x09$
0x45	1010	x	x	00	0x48	SALTO INCONDICIONAL
0x46	0100	1	0	00	0x09	$AC \leftarrow (AC) + 0x09$
0x47	0100	1	0	00	0x06	$AC \leftarrow (AC) + 0x06$
0x48	0101	0	1	11	0x10	$0x10 + I_{x3} \leftarrow (ACC)$
0x49	0100	0	0	11	0x10	$AC \leftarrow (AC) + (0x10 + I_{x3})$
0x4A	0101	0	0	11	0x10	$0x10 + I_{x3} \leftarrow (AC)$
0x4B	0011	x	x	10	xxxx	$I_{x2} + 1$
0x4C	0011	x	x	11	xxxx	$I_{x3} + 1$
0x4D	0110	x	x	01	xxxx	$I_{x1} - 1$
0x4E	0110	x	x	01	xxxx	$I_{x1} - 1$
0x4F	0111	x	x	01	0x	SALTO COND SI $I_x = 0$
0x50	1010	x	x	00	0x35	SALTO INCONDICIONAL
0x51	0000	x	x	xx	xxxx	FIN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
1	0	0	0	1	1	0	0	0	1	1	0	0	0	0
P_3				P_2				P_1				P_0		

$$C_0 = 1 \oplus 3 \oplus 5 \oplus 7 \oplus 9 \oplus 11 \oplus 13 \oplus 15$$

$$= 0 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$C_1 = 2 \oplus 3 \oplus 6 \oplus 7 \oplus 10 \oplus 11 \oplus 14 \oplus 15$$

$$= 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$C_2 = 4 \oplus 5 \oplus 6 \oplus 7 \oplus 12 \oplus 13 \oplus 14 \oplus 15$$

$$= 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 = 1$$

$$C_3 = 8 \oplus 9 \oplus 10 \oplus 11 \oplus 12 \oplus 13 \oplus 14 \oplus 15$$

$$= 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 1 = 1$$

	P_3	P_2	P_1	P_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

ERRORES EN POSICIÓN $1111_2 = 15$

MENSAJE CORREGIDO: 000011000011000

SIN BITS DE PARIDAD: 00001100010

LOS PASO A BASE 9:

$$0 \cdot 2^{10} + 0 \cdot 2^9 + 0 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 =$$

$$= 98_{10}$$

$$\begin{array}{r} 98_{10} \div 9 \\ 10 \text{ L } 8 \\ \underline{90} \\ 8 \end{array}$$

$$98_{10} = 118_9$$

PASAJE DIRECTO: $3^2 = 9$ CADA DÍGITO BASE 9 SE EXPRESA CON 2 DÍGITOS EN BASE 3

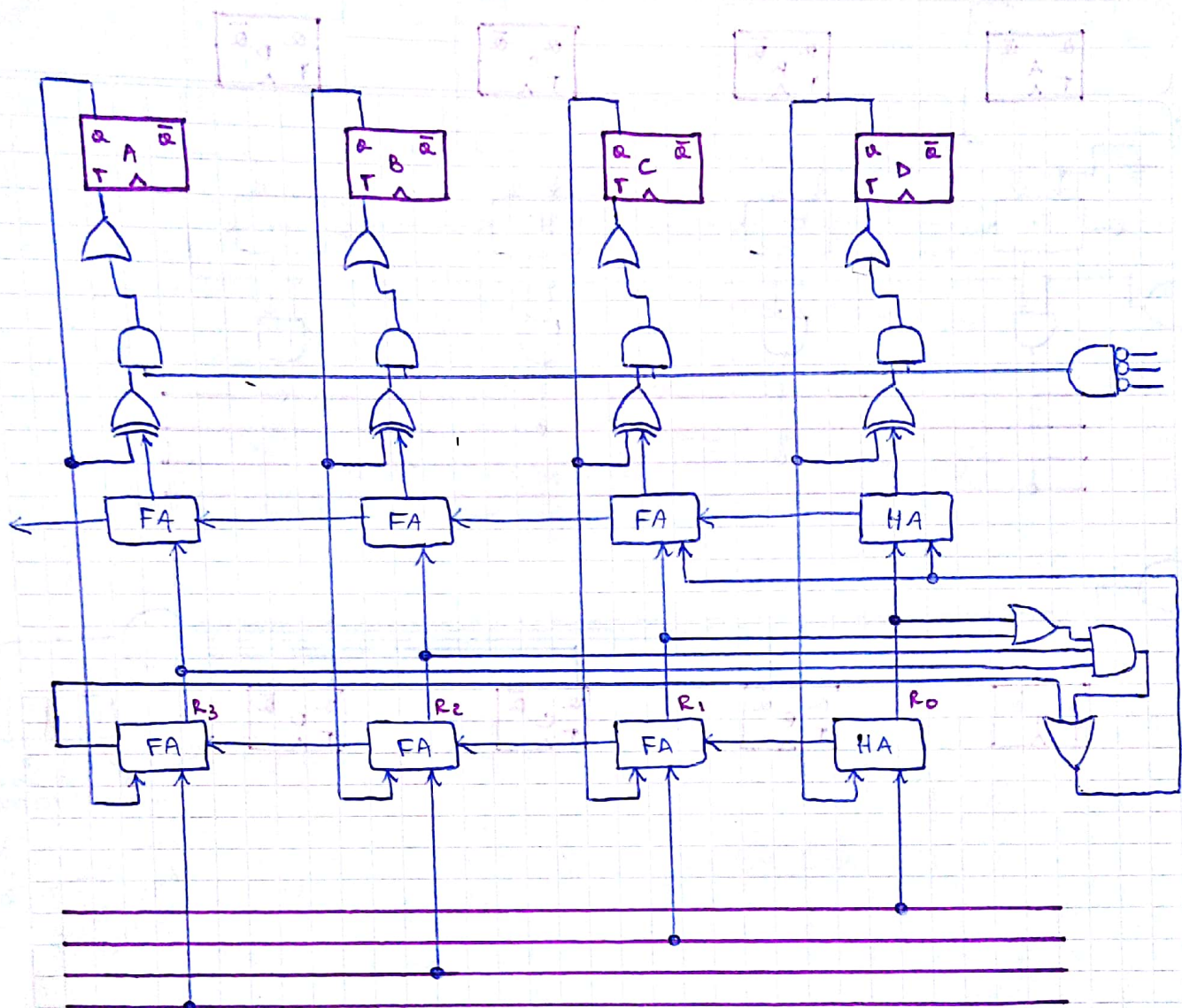
$$\begin{array}{c} 1 \quad 1 \quad 8 \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ 0 \quad 1 \quad 0 \quad 1 \quad 2 \quad 2 \end{array}$$

COMPROBACIÓN:

$$\begin{array}{r} 98_{10} \div 3 \\ 32 \text{ L } 3 \\ \underline{96} \\ 02 \end{array}$$

$$98_{10} = 10122_3$$

3



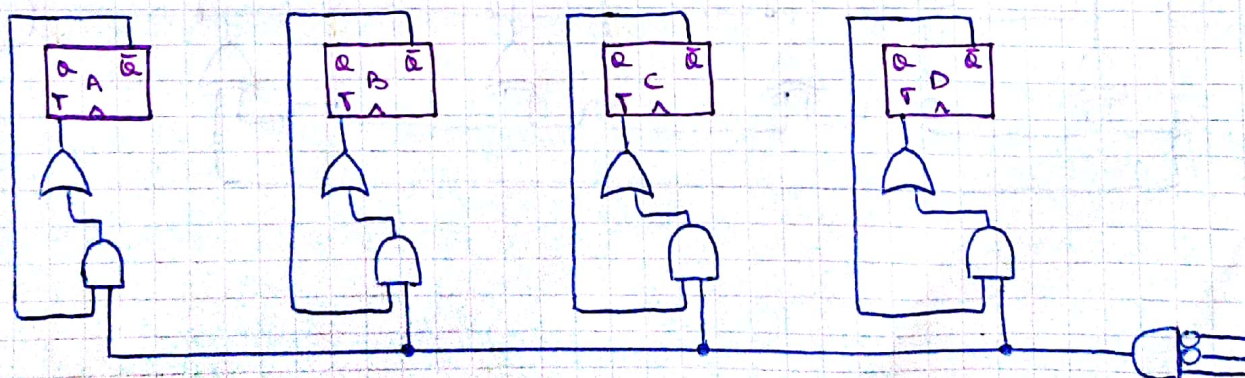
$R_3 \ R_2 \ R_1 \ R_0$
 13 1 1 0 1
 14 1 1 1 0
 15 1 1 1 1

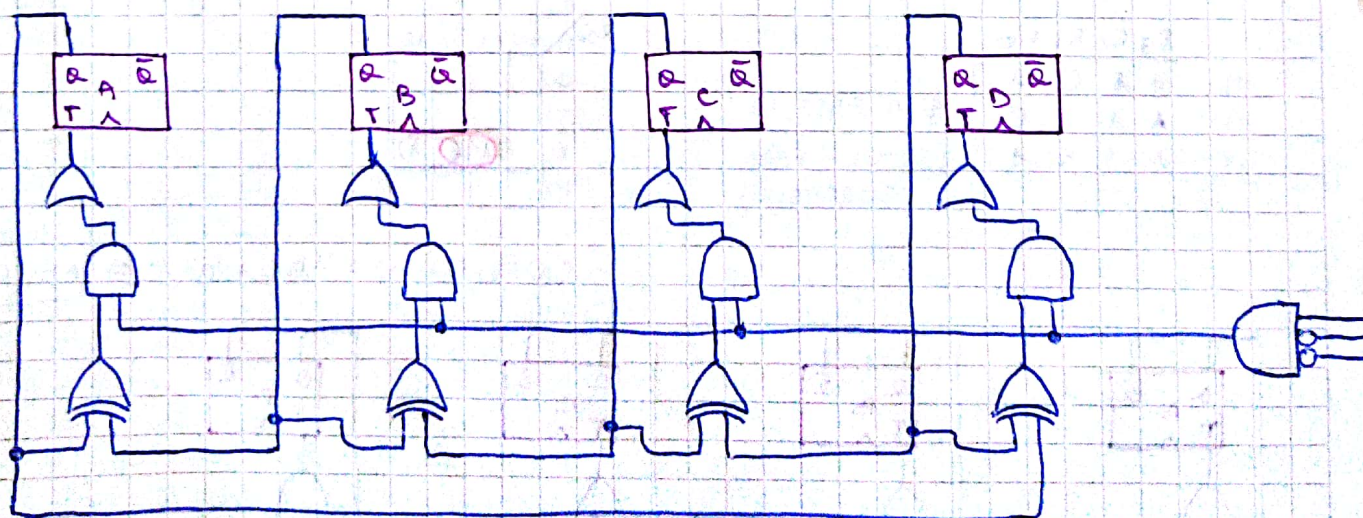
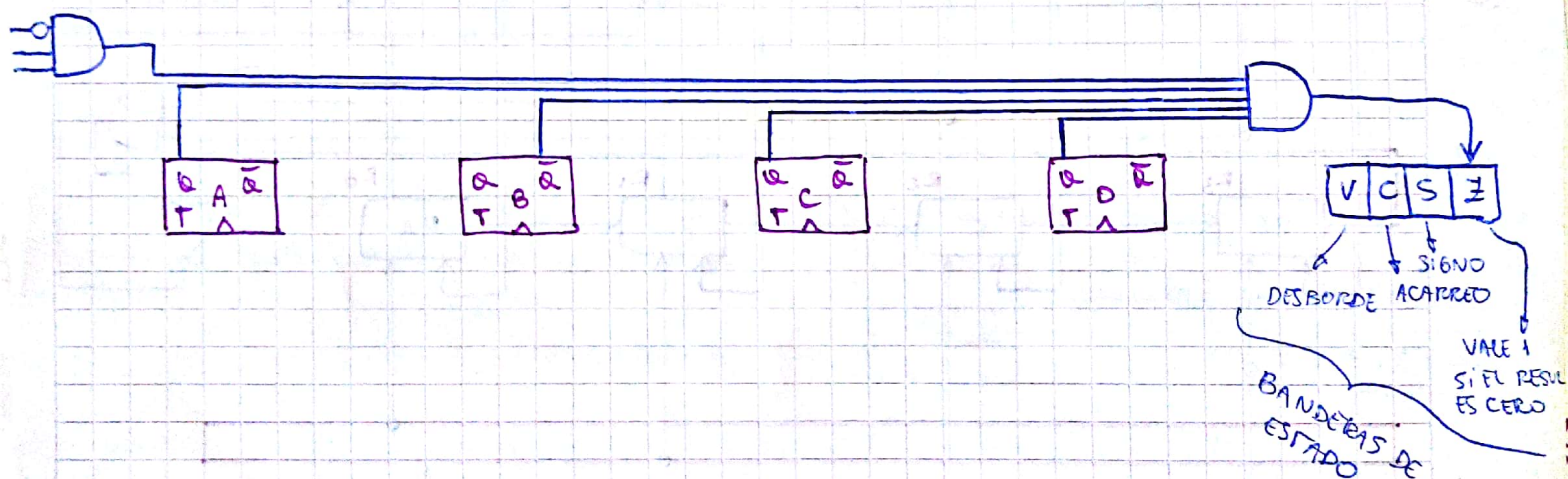
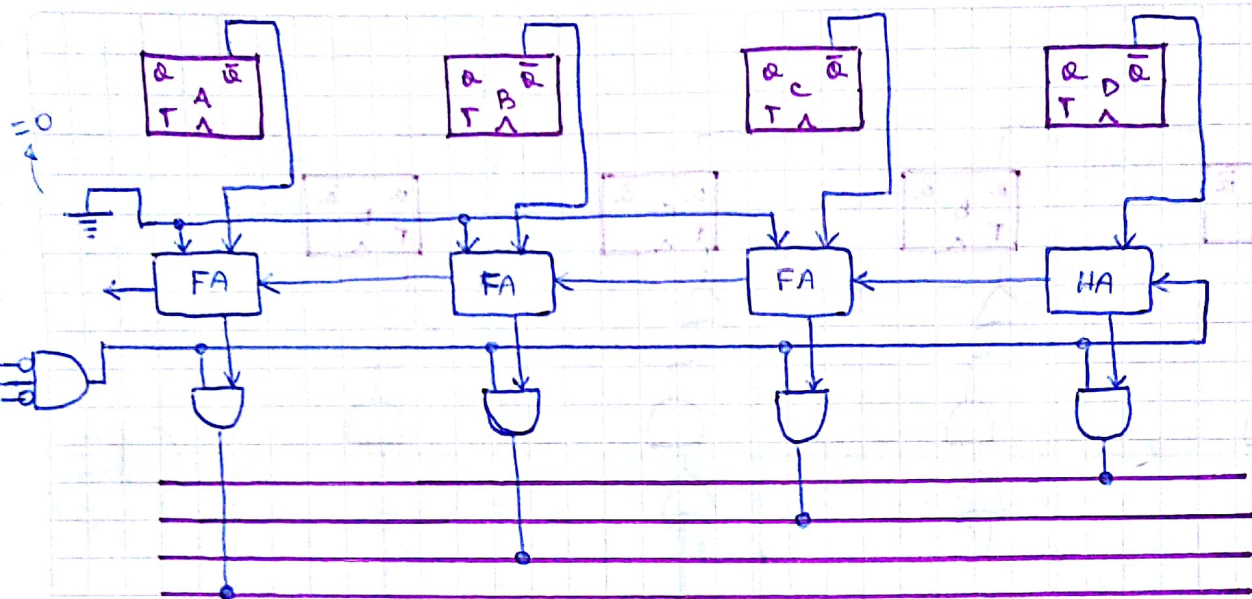
} $\notin$ A BASE 13
 (SOMA 3 PARA CORRIGIR)

R_{10}
 R_{32}

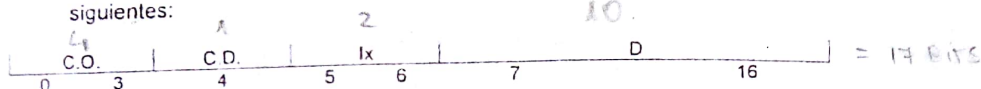
	00	01	11	10
00				
01				
11			1	1
10				

$$\text{CORRECCIÓN} = R_3 f_2 R_0 + R_3 R_2 R_1 = \boxed{R_3 R_2 (R_0 + R_1)}$$





1. (40%) Realice un programa en Lenguaje de Máquina que recorra el intervalo de memoria [0x1C; 0x60] y mueva los contenidos que son distintos a 0xFA (11111010), al intervalo que comienza a partir de la dirección 0x71. (incluya el diagrama de flujo o algoritmo correspondiente). Escriba todas las direcciones en hexadecimal. Las características de la máquina son las siguientes:



C.D.: 0 = Inmediato.
1 = Directo.

Ix: 00 = No direcciona reg. índice.
01 = Ix1.
10 = Ix2.
11 = Ix3.

Set de instrucciones:

C.O.	CD	Ix	D	Significado
0000	x	xx	x.....x	Fin.-
0001	x	xx	x.....x	Cargar AC.-
0010	x	xx	x.....x	Cargar Ix.-
0011	x	xx	x.....x	Almacenar (AC).-
0100	x	xx	x.....x	Salto condicional si (Ix)=0.-
0101	x	xx	x.....x	Salto incondicional.-
0110	x	xx	x.....x	Decrementar (Ix).-
0111	x	xx	x.....x	Salto condicional si (AC)=0.-
1000	x	xx	x.....x	AND con (AC)
1001	x	xx	x.....x	XOR con (AC)
1010	x	xx	x.....x	Restar al (AC)
1011	x	xx	x.....x	Incrementar (Ix).-

2. (15%) Completar la siguiente tabla y determinar el tamaño de la memoria y palabra de memoria.

	Decodificadores		Puntos de memoria		Lectura		Escritura		Totales	
	AND	Entradas	AND	Entradas	AND	Entradas	AND	Entradas	AND	Entradas
Mem. 2 D	32768	491520	2359296	6291456	24	48	0	0	2392088	6783024
Mem. 2 1/2 D	384	2944	2359296	6291456	3096	6192	128	256	2362904	6300848
Mem. 3 D	384	2944	2359296	8650752	24	48	0	0	2359704	8653744

3. (45%) Un circuito secuencial tiene dos entradas y dos salidas. Las entradas (X1, X2) representan un número en binario natural de 2 bits, N. Si el valor presente de N es mayor que el valor inmediatamente anterior, entonces la salida Z1 = 1. Si dicho valor es menor, entonces la salida Z2 = 1.

En cualquier otro caso, Z1 = Z2 = 0 (cero).

Realizar:

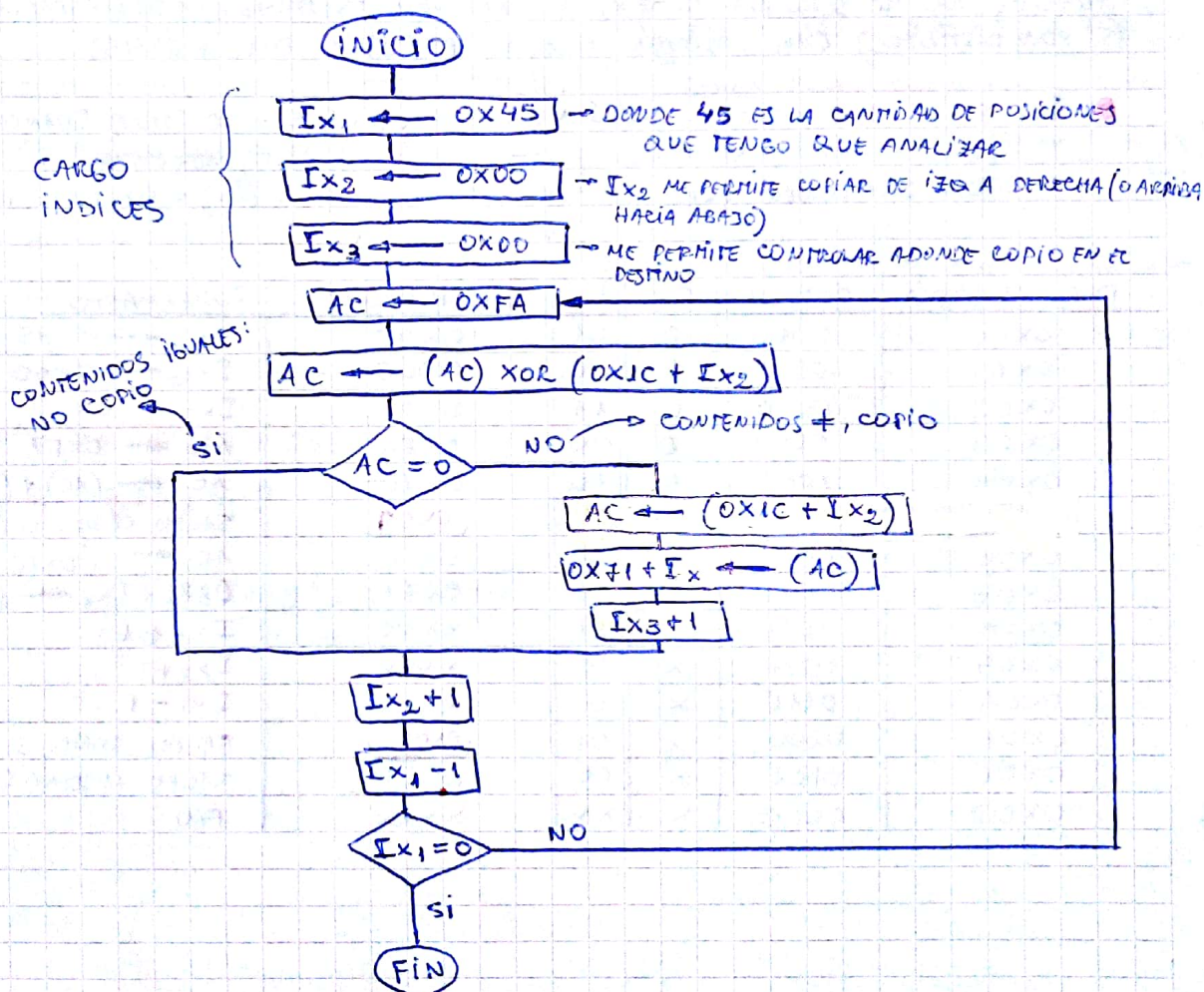
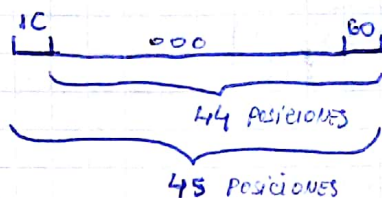
- Diagrama de estados.
- Tabla.
- Simplificaciones
- Círculo con biestables JK.

FINAL 01/08/12

PARA DETECTAR SI SON IGUALES O DISTINTOS LOS CONTENIDOS DE LAS POSICIONES A 0XFA (1111010) HAGO XOR ENTRE 0XFA Y EL(AC). SI SON IGUALES TODOS LOS BITS, (AC) = 0, SINO VA A SER MAYOR A CERO.

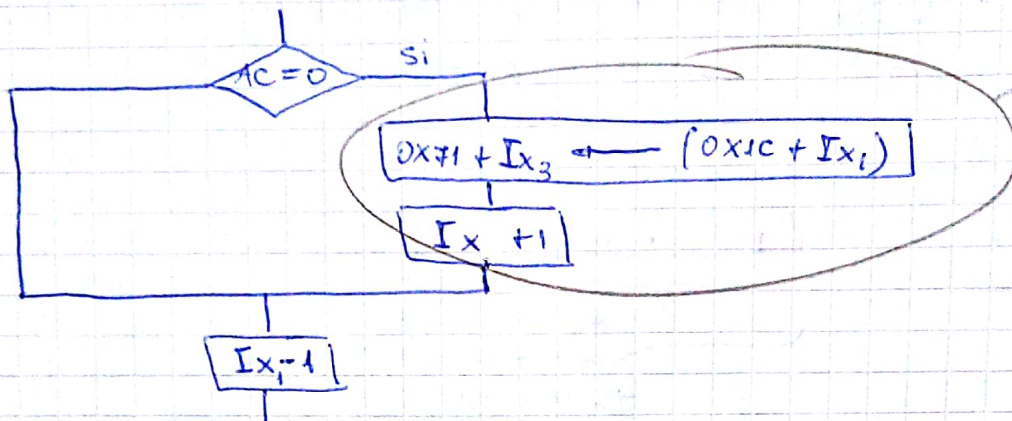
ANALITUD DEL INTERVALO

$$\begin{array}{r} 5'10 \\ - 1'0 \\ \hline 4'4 \end{array}$$



- EL HECHO DE TENER SOLO "SALTO CONDICIONAL SI (Ix) = 0" ME OBLIGA A CARGAR 45 EN EL Ix1, LO QUE ME ASEGURA HABER LA VERIFICACIÓN DE IGUALDAD 45 VECES, QUE ES LA CANTIDAD DE POSICIONES A ANALIZAR.
- SI TUVIERA "SALTO CONDICIONAL SI (Ix) < 0" ENTONCES PUEDO CARGAR 44 EN Ix1 Y AL FINAL CONTROLAR SI Ix < 0 EN LUGAR DE Ix = 0, Y LA CANTIDAD DE VECES QUE COMPARO ES EL MISMO.
- Ix1 E Ix2 SON NECESARIOS (UNO PARA IR DESCONTANDO ~~XXXX~~ HASTA DETECTAR 0 Y EL OTRO PARA IR INCREMENTANDO Y RECORRER EL INTERVALO EN ORDEN NORMAL) SI QUIERO COPIAR EN ORDEN NORMAL Y NO INVERSO, AUNQUE EL PROBLEMA

NO ESPECIFICA EL ORDEN DE COPIADO, SI DE $AC \rightarrow GO$ O DE $AC \leftarrow GO$. SI NO IMPORTA EL ORDEN, PUEDO USAR SOLO I_{x1} Y HACER



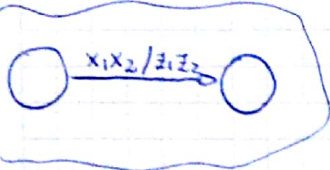
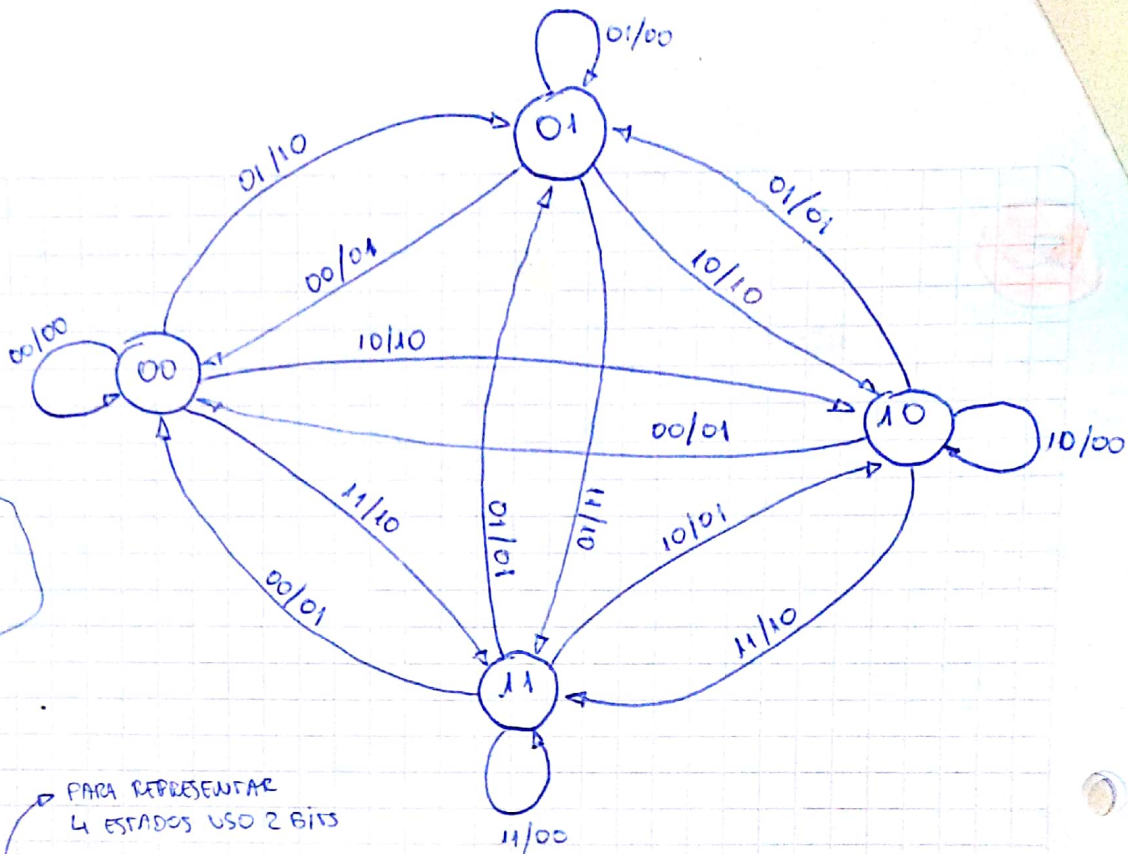
AUNQUE NO ME SERVIRÍA SI I_{x1} AL PRINCIPIO ES $0x45$, YA QUE ESTARÍA COPIANDO (SI SON DISTINTOS) UNA POSICIÓN QUE NO ES (OSEA $0x1C + 0x45$)

- 6 I_{x1} SE VA DESCONTANDO PORQUE ES LA ÚNICA FORMA DE SABER CUANDO RECORRIMOS TODO EL INTERVALO (X8 TENGO SALTO COND. SI $(I_x) = 0$), ~~0x00~~. SI LO INCREMENTARA TENDRÍA QUE TENER ALGO COMO "SALTO COND. SI $(I_x) = 0x45$)

DIRE. MEMORIA	COD. OP.	CD.	I_x	DIRE. IMPLICADA	COMENTARIO
0x00	0010	0	01	0x45	$I_{x1} \leftarrow 0x45$
0x01	0010	0	10	0x00	$I_{x2} \leftarrow 0x00$
0x02	0010	0	11	0x00	$I_{x3} \leftarrow 0x00$
0x03	0001	0	00	0xFA	$AC \leftarrow 0xFA$
0x04	1001	1	10	0x1C	$AC \leftarrow (AC) + (0x1C + I_{x2})$
0x05	0111	X	00	<u>0x09</u>	SALTO COND. SI $(AC) = 0$
0x06	0001	0	10	0x1C	$AC \leftarrow (0x1C + I_{x2})$
0x07	0011	0	11	0x71	$0x71 + I_{x3} \leftarrow (AC)$
0x08	1011	X	11	xxxx	$I_{x3} + 1$
0x09	1011	X	10	xxxx	$I_{x2} + 1$
0x0A	0110	X	01	xxxx	$I_{x1} - 1$
0x0B	0100	X	01	<u>0x0D</u>	SALTO COND. SI $I_{x1} = 0$
0x0C	0101	X	00	0x03	SALTO INCONDICIONAL
0x0D	0000	X	xx	xxxx	FIN

3

a)



b)

PARA REPRESENTAR
4 ESTADOS USO 2 BITS

ENTRADAS		ESTADOS		EST. SIGUIENTE		SALIDAS		FLIP FLOPS			
x_1	x_2	A	B	A'	B'	z_1	z_2	J_A	K_A	J_B	K_B
0	0	0	0	0	0	0	0	0	X	0	X
0	0	0	1	0	0	0	1	0	X	X	1
0	0	1	0	0	0	0	1	X	1	0	X
0	0	1	1	0	0	0	1	X	1	X	1
0	1	0	0	0	1	1	0	0	X	1	X
0	1	0	1	0	1	0	1	0	X	X	0
0	1	1	0	0	1	0	1	X	1	1	X
0	1	1	1	0	1	0	1	X	1	X	0
1	0	0	0	1	0	1	0	1	X	0	X
1	0	0	1	1	0	1	0	1	X	X	1
1	0	1	0	1	0	0	0	X	0	0	X
1	0	1	1	1	0	0	1	X	0	X	1
1	1	0	0	1	1	1	0	1	X	1	X
1	1	0	1	1	1	1	0	1	X	X	0
1	1	1	0	1	1	1	0	X	0	1	X
1	1	1	1	1	1	0	0	X	0	X	0

$Q(t)$	$Q(t+1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

c)

$x_1x_2 \backslash AB$	00	01	11	10
00			X	X
01			X	X
11	1	1	X	X
10	1	1	X	X

$$J_A = x_1$$

$x_1x_2 \backslash AB$	00	01	11	10
00		X	X	
01	1	X	X	1
11	1	X	X	1
10		X	X	

$$J_B = x_2$$

$x_1x_2 \backslash AB$	00	01	11	10
00				
01		1		
11	1	1		
10	1	1		

$$z_1 = x_1\bar{A} + x_1x_2\bar{B} + x_2\bar{A}\bar{B} = x_2\bar{B}(x_1 + \bar{A}) + x_1\bar{A}$$

$x_1x_2 \backslash AB$	00	01	11	10
00	X	X	1	1
01	X	X	1	1
11	X	X		
10	X	X		

$$K_A = \bar{x}_1$$

$x_1x_2 \backslash AB$	00	01	11	10
00	X	1	1	X
01	X			X
11	X			X
10	X	1	1	X

$$K_B = \bar{x}_2$$

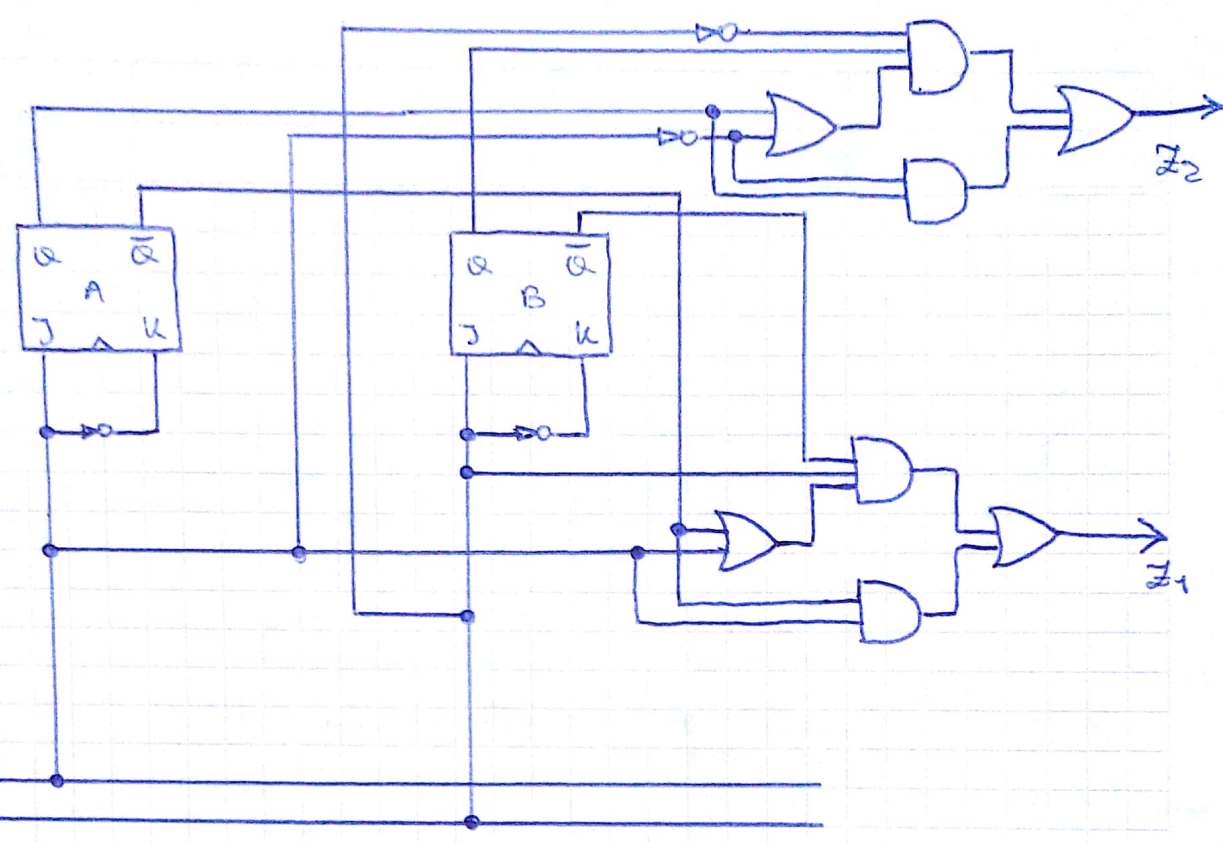
$x_1x_2 \backslash AB$	00	01	11	10
00			1	1
01		1		
11				
10				1

$$z_2 = \bar{x}_1A + \bar{x}_2AB + \bar{x}_1\bar{x}_2B = \bar{x}_2B(A + \bar{x}_1) + \bar{x}_1A$$

cuito r
lidas
mb
f

d)

x_1
 x_2



ARQUITECTURA DE COMPUTADORAS

EXAMEN FINAL 12/07/12

EJERCICIO 1: (40%) Una máquina expendedora de gaseosas está controlada por un circuito secuencial. Dicho circuito recibe como entradas el código de dos bits (tabla 1) de las monedas que va introduciendo el usuario. Como salidas, el circuito debe activar 2 señales, correspondientes a si se debe o no entregar la gaseosa (G), y la cantidad de cambio a reintegrar (C1 y C0), codificadas según las tablas 2 y 3 respectivamente.

El funcionamiento del circuito es el siguiente: El usuario debe introducir monedas hasta alcanzar la cantidad de \$1 (valor de la gaseosa). Una vez que la suma de monedas alcance dicho importe, automáticamente la máquina entrega la gaseosa, retornando además el cambio correspondiente. Por ejemplo, si el usuario ha introducido 1 moneda de 25 y 2 monedas de 50, la máquina deberá entregar la gaseosa y una moneda de 25 (salidas $C1\ C0 = 1\ 0\ 1$)

Tabla 1-entradas
Codificación de la entrada M

M1	M2	Significado
0	0	No hay moneda
0	1	Nueva moneda de \$0.25
1	0	Nueva moneda de \$0.50
1	1	Nueva moneda de \$1

Tabla 2-salida
Codificación de la salida G

G	Significado
0	No entregar
1	Entregar Gaseosa

Tabla 3-salidas
Codificación del cambio C

C1	C0	Significado
0	0	No hay cambio
0	1	Cambio de 0.25
1	0	Cambio de 0.50
1	1	Cambio de 0.75

EJERCICIO 2: (40%) Las direcciones 0x2E y 0x2F contienen números en base 13.

Por ejemplo: (0x2E)=0000xxxx (xxxx representa un número válido en base 13)

Realice un programa en assembler para microcontroladores PIC 16c84 que sume los contenidos de las direcciones 0x2E y 0x2F almacenando el resultado (en base 13) en la dirección 0x2A. Se pide:

- Programa en Assembler.
- Contenidos de W, FSR y PCL al finalizar la ejecución del programa.
- Contenidos de las siguientes posiciones de memoria RAM al finalizar la ejecución del programa, cuyos valores antes de correr el programa son
(0x2E) = 0x09 (0x2F) = 0x0A (0x2A) = 0x13

EJERCICIO 3: (20%) Para una máquina con arquitectura Von Neumann, determinar cuantos bits tendrán cada uno de los buses y registros. Calcular compuertas AND y Entradas si la memoria tiene una arquitectura 2 1/2 D para el decodificador, MC, lectura y escritura. Justificar.

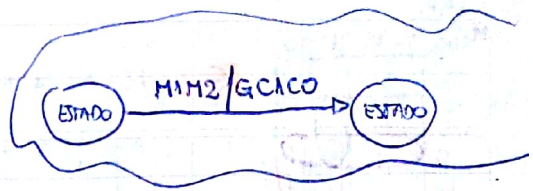
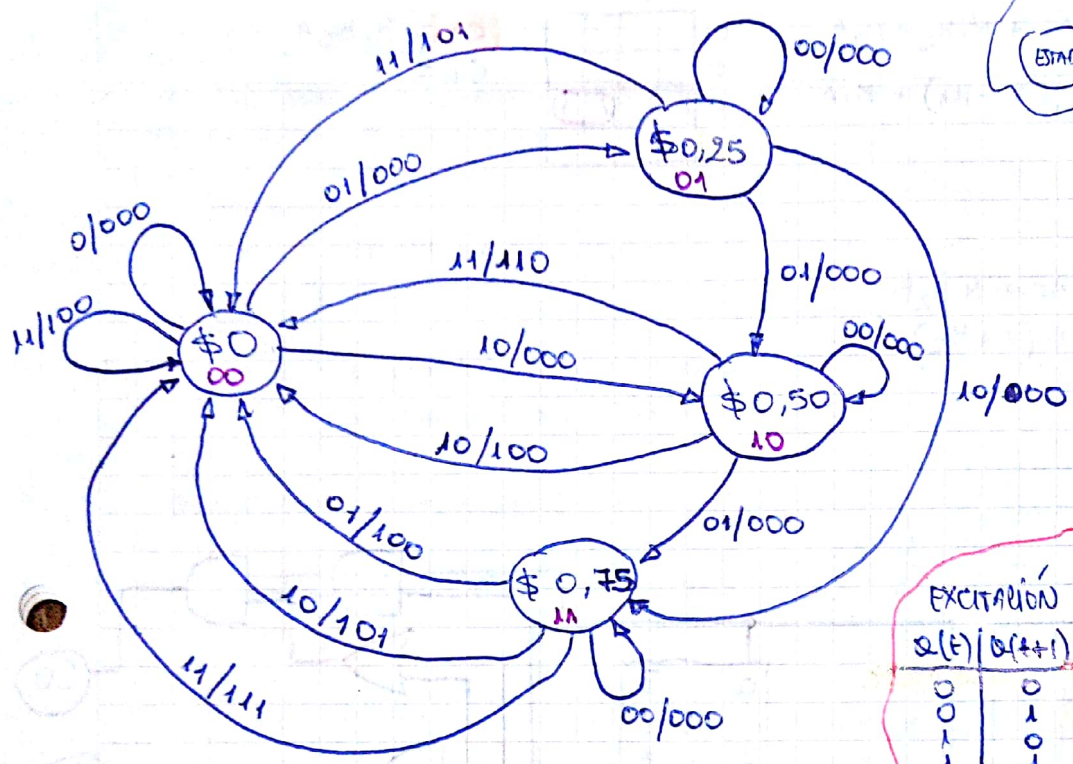
DATOS:

- Memoria de instrucciones 64 Kpalabras de 8K
- Repertorio de instrucciones= 32
- Tipos de direccionamiento: directo, inmediato e indirecto.

Julio 2015

JULIO 2015

FINAL 12/07/12



ENTRADAS $\begin{cases} M1 \\ M2 \end{cases} \rightarrow 2 \text{ Bits}$

ESTADOS = 4 $\rightarrow$ USO 2 BITS

ENT + ESTADOS = 4 BITS
 $2^4 = 16$ FILAS EN LA TABLA

EXCITACIÓN

$Q(t)$	$Q(t+1)$	J	K
0	0	0	X
0	1	X	X
1	0	X	1
1	1	X	0

ENTRADAS		ESTADOS		ESTADOS SIGUIENTES		SALIDAS			FLIP FLOPS			
M_1	M_2	A	B	A'	B'	$G1$	$C1$	$C0$	J_A	K_A	J_B	K_B
0	0	0	0	0	0	0	0	0	0	X	0	X
0	0	0	1	0	1	0	0	0	0	X	X	0
0	0	1	0	1	0	0	0	0	X	0	0	X
0	0	1	1	1	1	0	0	0	X	0	X	0
0	1	0	0	0	1	0	0	0	0	X	1	X
0	1	0	1	1	0	0	0	0	1	X	X	1
0	1	1	0	1	1	0	0	0	X	0	1	X
0	1	1	1	0	0	1	0	0	X	1	X	1
1	0	0	0	1	0	0	0	0	1	X	0	X
1	0	0	1	1	1	0	0	0	1	X	X	0
1	0	1	0	0	0	1	0	0	X	1	0	X
1	0	1	1	0	0	1	0	1	X	1	X	1
1	1	0	0	0	0	1	0	0	0	X	0	X
1	1	0	1	0	0	1	0	1	0	X	X	1
1	1	1	0	0	0	1	1	0	X	1	0	X
1	1	1	1	0	0	1	1	1	X	1	X	1

$J_A = \overline{M_1} M_2 B + M_1 \overline{M_2}$

$M_1 \backslash M_2$	00	01	11	10
00			X	X
01		X	X	X
11			X	X
10	1	1	X	X

$J_B = \overline{M_1} M_2$

$M_1 \backslash M_2$	00	01	11	10
00			X	X
01	1	X	X	1
11		X	X	
10		X	X	

$K_A = M_1 + M_2 B$

$M_1 \backslash M_2$	00	01	11	10
00	X	X		
01	X	X	1	
11	X	X	1	1
10	X	X	1	1

$K_B = M_2 + M_1 A$

$M_1 \backslash M_2$	00	01	11	10
00	X			X
01	X	1	1	X
11	X	1	1	X
10	X		1	X

M ₁ M ₂ \ AB	00	01	11	10
00				
01			1	
11	1	1	1	1
10			1	1

$$G = M_2 AB + M_1 M_2 + M_1 A$$

$$M_2 (AB + M_1) + M_1 A$$

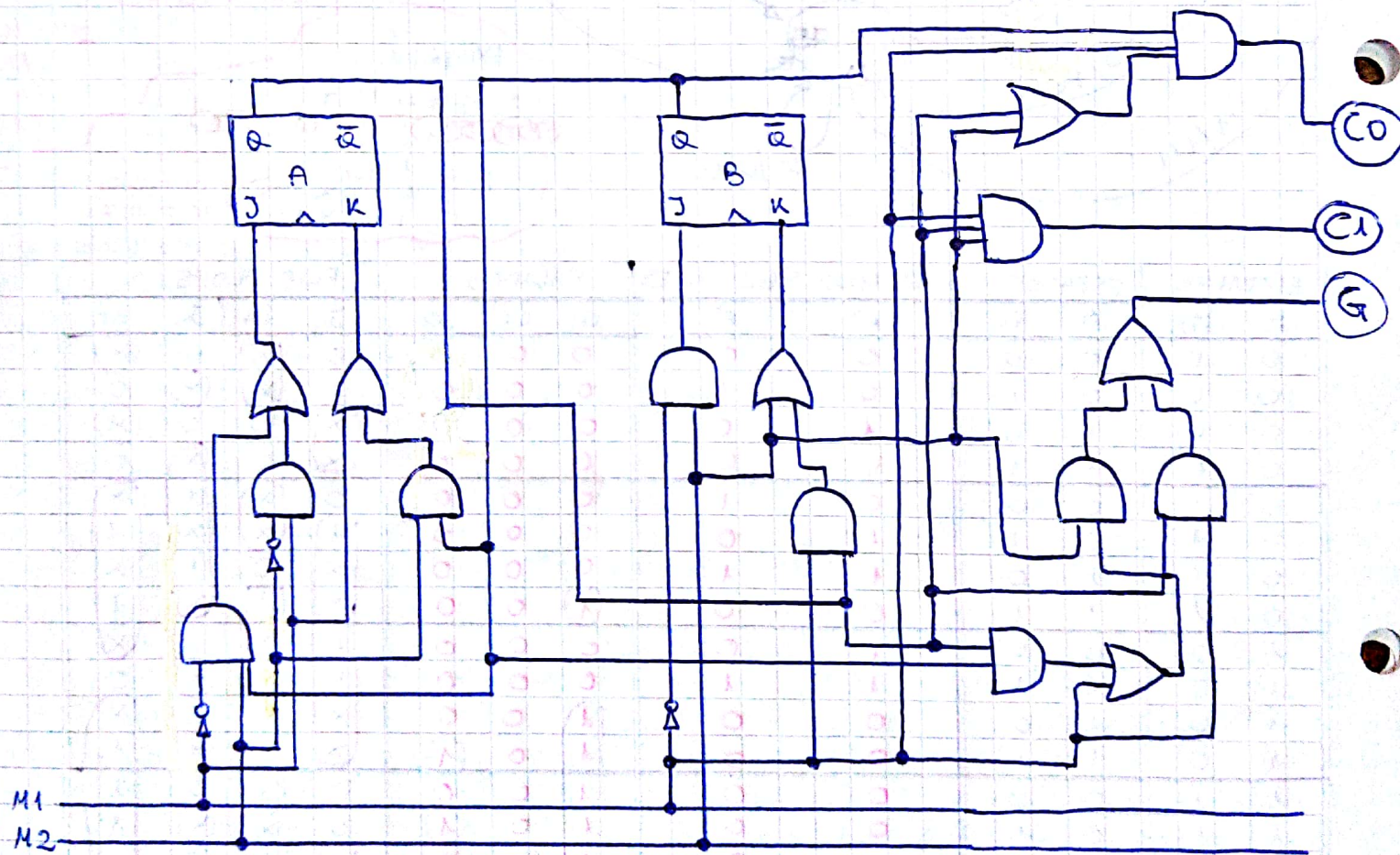
M ₁ M ₂ \ AB	00	01	11	10
00				
01				
11			1	1
10				

$$C_1 = M_1 M_2 A$$

M ₁ M ₂ \ AB	00	01	11	10
00				
01				
11	1	1		
10				1

$$C_0 = M_1 AB + M_1 M_2 B$$

$$M_1 B (A + M_2)$$



2

BASE 13 = DE "0" A "C" (12)

LO QUE HACEMOS ES:

AL ~~10000~~ RESULTADO QUE OBTENGO DE LA SUMA LE SUMO 0X03, PORQUE

$$\begin{array}{r} 0000 \\ \vdots \\ 1100 \end{array} \left. \vphantom{\begin{array}{r} 0000 \\ \vdots \\ 1100 \end{array}} \right\} \text{VÁLIDO EN BASE 13}$$

$$\begin{array}{r} 1101 \\ 1110 \\ 1111 \end{array} \left. \vphantom{\begin{array}{r} 1101 \\ 1110 \\ 1111 \end{array}} \right\} \text{NO VÁLIDO EN BASE 13, PARA CORREGIR DEBO SUMAR 0011(3)}$$

POR EJEMPLO TENGO:

$$\begin{array}{r} 00 \\ + 03 \\ \hline 10 \end{array}$$
 RESULTADO
 NO VÁLIDO EN BASE 13

SI HAY DE (ACARREO INTERMEDIO) QUIERE DECIR QUE EL DÍGITO DE MENOR PESO DEL RESULTADO NO ESTABA EN BASE 13 (COMO EN EL EJEMPLO), OSEA:

$$\begin{array}{r} 000001101 \\ + 000000011 \\ \hline 000001111 \end{array}$$
 (ACARREO DESDE EL BIT 3 O BIEN CUARTO BIT)

DESPUES COMPRUEBO EL DÍGITO DE MAYOR PESO SUMANDO 0X30, Y SI OBTENGO C (ACARREO) ENTONCES QUIERE DECIR QUE TAMPOCO ESTABA EN BASE 13.

POR EJEMPLO, EL RESULTADO ORIGINAL ME DA 0XCD:

$$\begin{array}{r} CD \\ + 03 \\ \hline 100 \end{array}$$
 CORREGIDO

$$\begin{array}{r} 100 \\ + 30 \\ \hline 130 \end{array}$$
 A BASE 13
 130
 CARREO
 NUMERO CORREGIDO

- TENER EN CUENTA QUE, COMO FABA ARRIBA, LA CORRECCIÓN DEL DÍGITO DE LA DERECHA (MENOR PESO), PUEDE HACER QUE EL DE LA IZQUIERDA PASE DE PERTENECER A BASE 13 A NO HABERLO (DEBIDO AL ACARREO)
- EN ASSEMBLER, LOS PROBLEMAS DE CORRECCIÓN DE BASES CASI SIEMPRE SE HACEN ASÍ, USANDO DC, C Y SUMANDO 0X03 Y 0X30, DONDE 3 ES LO QUE DEBO SUMAR PARA CORREGIR EN ESTA BASE

RECORDAR: TANTO EN SUBWF COMO SUBLW SI EL BIT 0 DEL REG. ESTADO (C = CARRY) ES

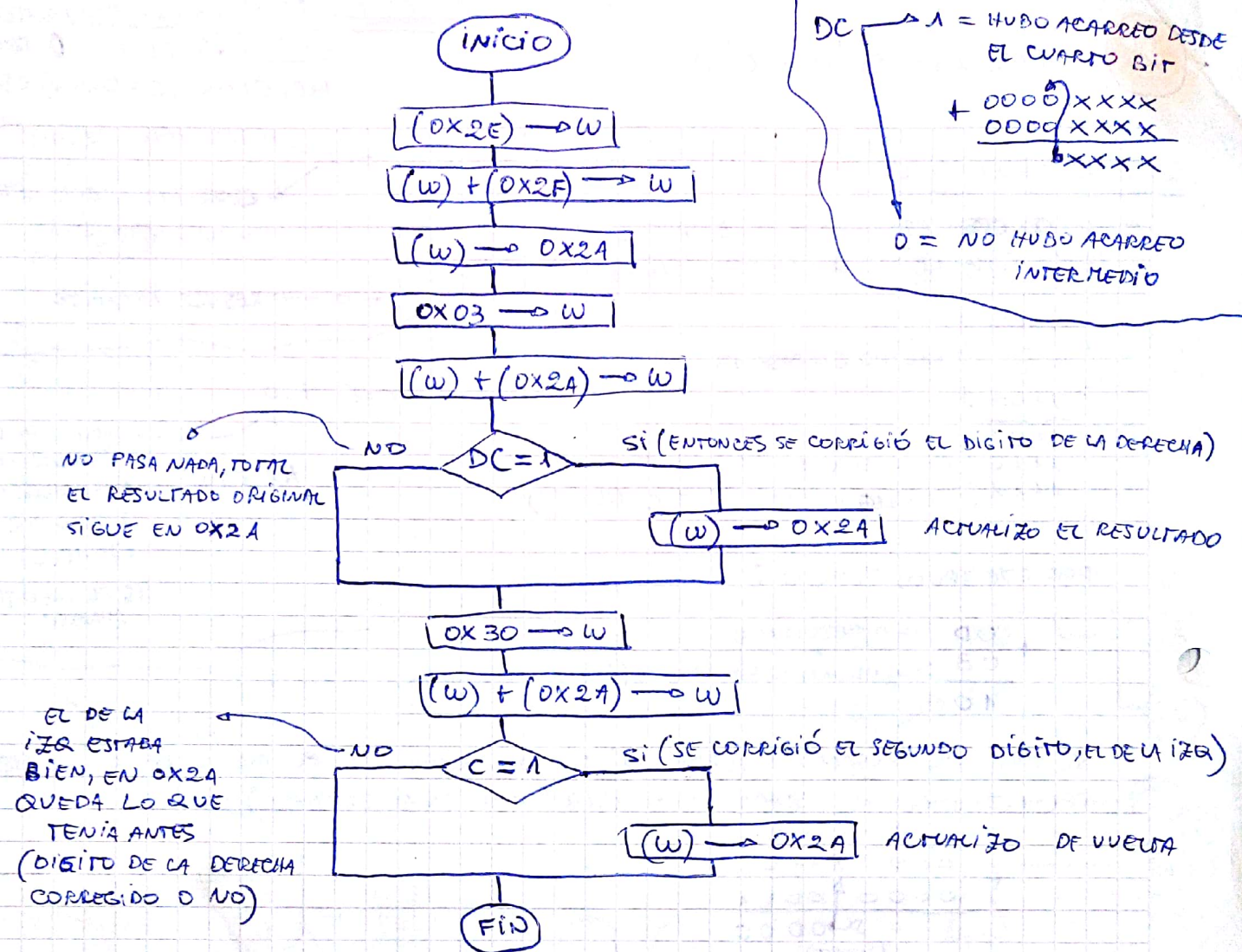
$= 1 \rightarrow$ RESULTADO POSITIVO
 $\rightarrow$ ~~0~~ RESULTADO CERO EN CUYO CASO Z=1

$= 0 \rightarrow$ RESULT NEGATIVO

MOV F, D

0 MUEVE A W

1 MUEVE A F MISMO (PARA COMPROBAR SI ES CERO POR EJEMPLO) ES EL PREDETERMINADO



LIST P = 16CB4

resul	equ 0x2A	org 0x00
0x00	movf 0x2E, 0	
0	addwf 0x2F, 0	
0	movwf resul	
0	movlw 0x03	
	addwf resul, 0	
	btfss STATUS, 1	
	movwf resul	
	movlw 0x30	
	addwf resul, 0	
0x09	btfss STATUS, 0	
0x0A	movwf resul	
	end	

; SALTAR LA SIG. INSTRUCCIÓN SI DC = 1

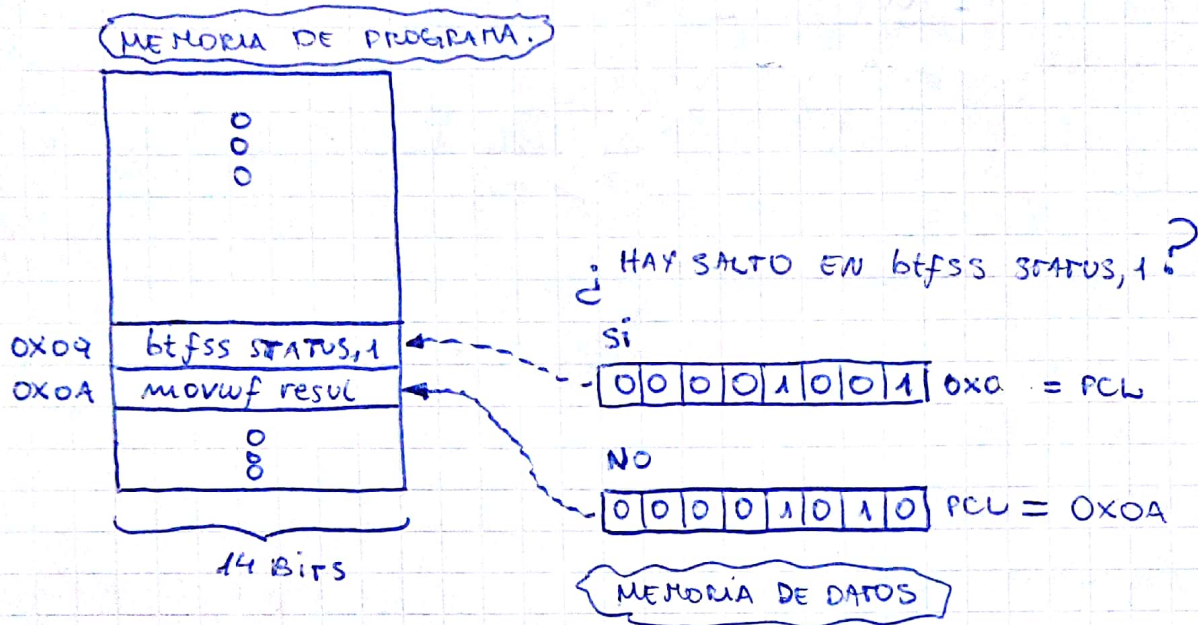
; SALTAR LA SIG. INSTRUCCIÓN SI C = 1

b) W puede quedar con un LITERAL donde los 2 dígitos están corregidos, o que solo el de la izquierda (y si no había que corregir, quedo con el dígito de la izquierda con +3).

FSR no se usa, por lo que queda en cero.

PCL tiene los 8 BITS DE MENOR PESO DEL PROGRAM COUNTER (PCL = 8 BITS, PC = 13 BITS). Por lo tanto PCL codifica hasta 256 instrucciones. En general

comenzando de 0x00 como dirección de la primera instrucción, PCL terminará conteniendo la dirección de la ~~última~~ última instrucción que se ejecute, en mi caso, si salta en el último BTFSS, ésta es la última que se ejecuta, por lo que (PCL) = 0x09 (que es la dirección en la memoria de programas del último BTFSS. Si no salta (PCL) = 0x0A, que corresponde a MOVWF RESL.



c) (0x2E) = 0x09 (no cambia)
 (0x2F) = 0x0A (" ")

$$\begin{array}{r}
 + 0x09 \\
 0x0A \\
 \hline
 0x13 \rightarrow \text{YA ES CORRECTO}
 \end{array}$$

PRIMERA COMPROBACIÓN:

$$\begin{array}{r}
 + 0x13 \\
 0x03 \\
 \hline
 0x16 \rightarrow \text{NO HAY DC, QUEDA 0x13}
 \end{array}$$

SEGUNDA COMPROBACIÓN:

$$\begin{array}{r}
 + 0x13 \\
 0x30 \\
 \hline
 0x43 \rightarrow \text{NO HAY C, QUEDA 0x13}
 \end{array}$$

(0x2A) = 0x13

EJERCICIO 1: (45%)

En el intervalo de memoria V1 [0x25;0x2F] se tienen cargados valores que representan errores sobre un conjunto de dispositivos. Los bits 0, 1, 2 y 3 indican la gravedad de las fallas que tuvo dicho dispositivo. El significado del tipo de error se interpreta de la siguiente manera:

Bit 0 está puesto a 1: Falla menor

Bit 1 está puesto a 1: Falla crítica recuperada automáticamente

Bit 2 está puesto a 1: Falla crítica recuperada por fuerza bruta

Bit 3 está puesto a 1: Falla Grave, dispositivo bloqueado

Consigna: Escriba un programa en assembler para microcontroladores PIC16c84 que verifique en el intervalo V1 las cargas que registran dispositivos que hayan tenido Fallas Graves y Crítica Recuperada por Fuerza Bruta. Guardar en V2 [0x15;0x1F] las direcciones de V1 que hayan verificado esa condición.

EJERCICIO 2: (45%)

Un sistema recibe secuencialmente datos de 1 bit a través de su entrada X.

Diseñe un circuito que dé salida Z=1 cuando se haya recibido X=1 durante tres o más intervalos de reloj consecutivos.

Un ejemplo del comportamiento que describe el enunciado es el siguiente:

X: 011001011110111110.....

Z: 000000000100011110.....

Realizar:

- Diagrama de estados.
- Tabla.
- Simplificaciones
- Circuito con biestables JK.

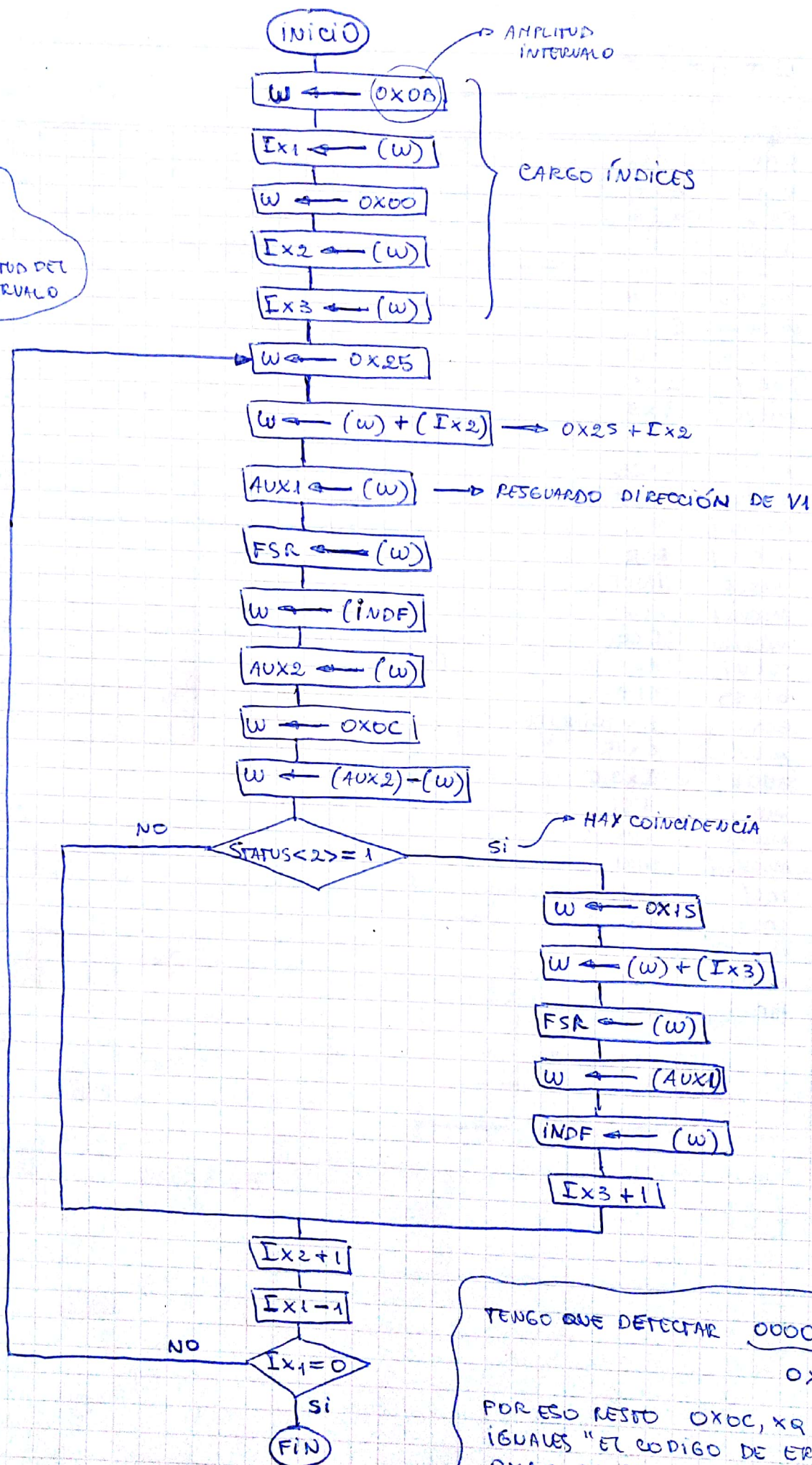
EJERCICIO 3: (10%)

- Codificar el número -52 en AIKEN, Exceso en 3, BCD zoneado y empaquetado con signo.
- Con el valor en obtenido en el código Aiken, aplicarle Hamming.

1

$$\begin{array}{r} 2F \\ - 25 \\ \hline + 0A \\ 1 \\ \hline 0B \end{array}$$

0B → AMPLITUD DEL INTERVALO



TENGO QUE DETECTAR 0000 11 00
0x0C

POR ESO RESTO 0x0C, XQ SI SON IGUALES "EL CODIGO DE ERROR" Y 0x0C, EL RESULTADO ME DA CERO Y STATUS<2> = 1, PORQUE ES EL BIT DE CERO.

LIST p = 16C84

;-----

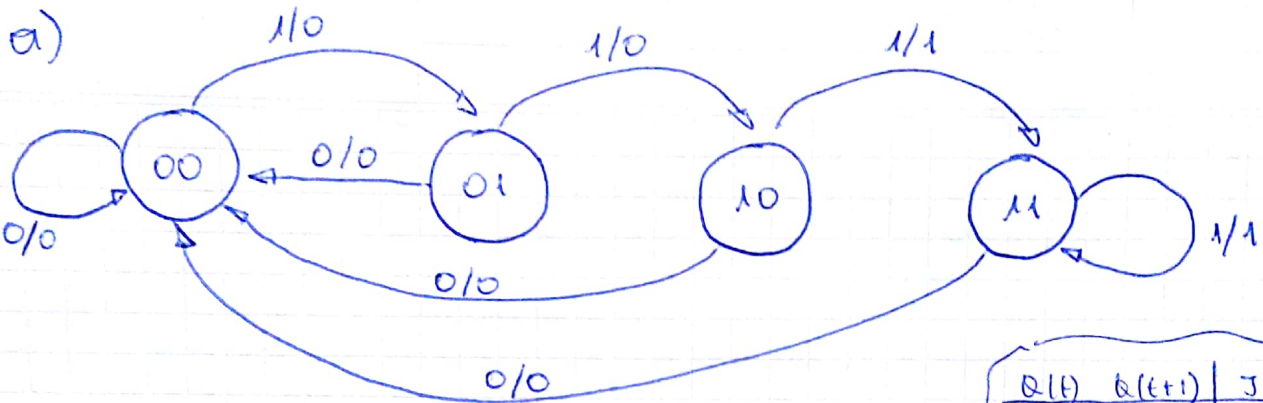
Ix1	equ	0x30
Ix2	equ	0x31
Ix3	equ	0x32
AUX1	equ	0x34
AUX2	equ	0x35

;-----

	movlw	0x0B
	movwf	Ix1
	movlw	0x00
	movwf	Ix2
	movwf	Ix3

BUCLE	movlw	0x25
	addwf	Ix2,0
	movwf	AUX1
	movwf	FSR
	movf	INDF,0
	movwf	AUX2
	movlw	0x0C
	subwf	AUX2,0
	btfss	STATUS,2
	GOTO	NOCOINCIDE
	movlw	0x15
	addwf	Ix3,0
	movwf	FSR
	movf	AUX1,0
	movwf	INDF
	incf	Ix3,1
NOCOINCIDE	incf	Ix2,1
	decfsz	Ix1,1
	GOTO	BUCLE
	fin	

2



Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

PARA 4 ESTADOS $\rightarrow$ 2 BITS

b)

ENTRADA	ESTADOS		ESTADOS SIG		SALIDA	FLIP-FLOPS			
X	A	B	A'	B'	Z	J _A	K _A	J _B	K _B
0	0	0	0	0	0	0	X	0	X
0	0	1	0	0	0	0	X	X	1
0	1	0	0	0	0	X	1	0	X
0	1	1	0	0	0	X	1	X	1
1	0	0	0	1	0	0	X	1	X
1	0	1	1	0	0	1	X	X	1
1	1	0	1	1	1	X	0	1	X
1	1	1	1	1	1	X	0	X	0

c)

AB

X	00	01	11	10
0			X	X
1		X	X	X

$J_A = XB$

AB

X	00	01	11	10
0	X	X	1	1
1	X	X		

$K_A = \bar{X}$

AB

X	00	01	11	10
0		X	X	
1	X	X	X	1

$J_B = X$

AB

X	00	01	11	10
0	X	1	1	X
1	X	1		X

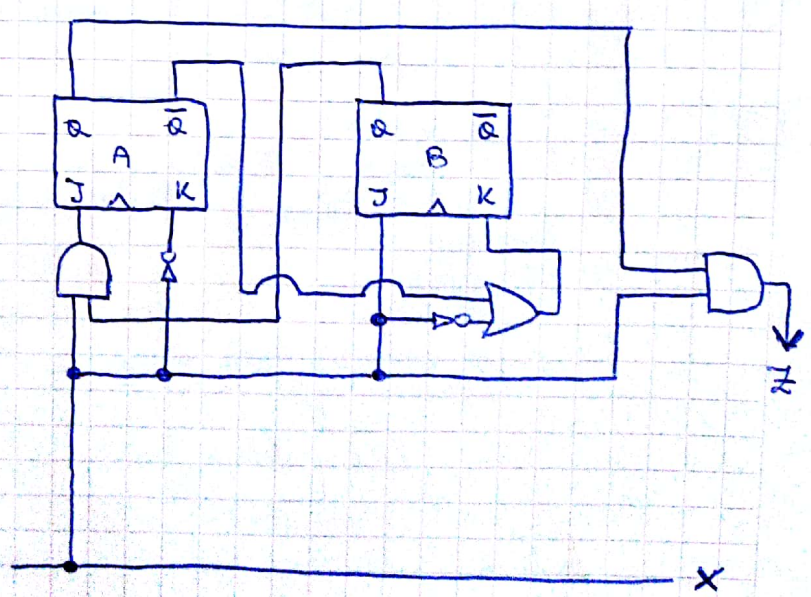
$K_B = \bar{A} + \bar{X}$

AB

X	00	01	11	10
0				
1			1	1

$Z = XA$

d)



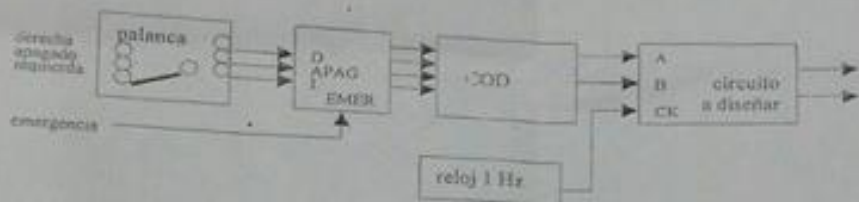
ARQUITECTURA DE COMPUTADORAS **EXAMEN FINAL 03-03-16**

EJERCICIO 1: (45%)

Se pretende diseñar el sistema de encendido de intermitencia de un coche. Para ello hay que diseñar un circuito secuencial que cumpla las siguientes especificaciones, de acuerdo con la figura adjunta.

Cuando la palanca se coloque en la posición DERECHA, se deberá encender y apagar de forma intermitente la luz identificada como D, de forma sincrónica con un reloj de 1 Hz. Cuando la palanca se coloque en la posición IZQUIERDA, se deberá encender y apagar de forma intermitente la luz identificada como I. Si la forma sincrónica con un reloj de 1 Hz. Cuando la palanca se coloque en la posición central (APAGADO) no se encenderá ninguna luz. Cuando se active el interruptor de EMERGENCIA, se activarán ambas luces simultáneamente, y se desactivarán ambas de forma sincrónica con el reloj, independientemente de la posición de la palanca, es decir, la entrada de emergencia tiene prioridad absoluta.

a) Diseñar el circuito adicional necesario para que nunca entre más de una señal activa al codificador y poner una tabla de salida del codificador para todos los casos posibles.
 b) Definir los estados, su codificación y su diagrama.
 c) Realizar la tabla de excitación usando biestables T.
 d) Implementar el circuito.



EJERCICIO 2: (10%) Codifique la siguiente información expresada en hexadecimal -95₁₆

- a) Aiken
 b) Basándose en el resultado del punto (2.a) codifique utilizando el método de Hamming. Explique los pasos realizados.

EJERCICIO 3: (45%) El intervalo de memoria [0x000, 0x059] de un CPU arquitectura Von Neumann almacena, en forma contigua, las lecturas de temperatura de un horno. Los valores corresponden a un día completo de lecturas que fueron tomadas y almacenadas cada de 15 minutos. Se desea conocer las horas del día en que la temperatura no se mantuvo constante. Realice un programa en lenguaje de máquina que almacene a partir de 0x7FF las horas del día en que hubo variación de temperatura. Las características de la máquina son las siguientes:

C.O.	C.D.	Acc	Ix	B	D
0	3	4	5	6	7
8					17

C.D.: 0 = Inmediato 1 = Directo	Ix: 0 = No direcciona reg. índice 1 = Ix1	Acc: 0 = No direcciona Acc 1 = Acc1
		B: 0 = No direcciona 1 = Base 1

Set de instrucciones:

C.O.	CD	Acc	Ix	B	D	Significado
0000	x	x	x	x	x	Fin.-
0001	x	x	x	x	x	Cargar AC.-
0010	x	x	x	x	x	Almacenar (AC).-
0011	x	x	x	x	x	Cargar Ix.-
0100	x	x	x	x	x	Cargar B.-
0101	x	x	x	x	x	Salto incondicional.-
0110	x	x	x	x	x	Salto condicional si (Ix) = 0.-
0111	x	x	x	x	x	Salto condicional si (AC) = 0.-
1000	x	x	x	x	x	Salto condicional si (AC) < 0.-
1001	x	x	x	x	x	XOR con (AC)
1010	x	x	x	x	x	Restar al (AC)
1011	x	x	x	x	x	Incrementar (Ix).-
1100	x	x	x	x	x	Decrementar (Ix).-

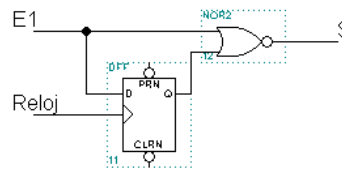
Comentarios Memoria: La última lectura de un horario es la primera del próximo

Franja horaria	Hora de lectura	Lecturas	Resultado
Hora 0	0000	360	Vano, almacena 0 en 1era posición de 2do intervalo
	0015	360	
	0030	360	
	0100	360	
Hora 1	0115	360	No vano
	0130	360	
	0200	360	
Hora 2	0215	360	Vario, almacena 2 en 2da posición del 2do intervalo
	0230	365	
	0300	360	

1. Minimización de las funciones e implementación del circuito

Simplificando por Karnaugh obtendremos la función de entrada al biestable D para el cálculo del nuevo estado, y la función de salida S.

$$D = E1$$
$$S = Q' E1' = (Q + E1)'$$

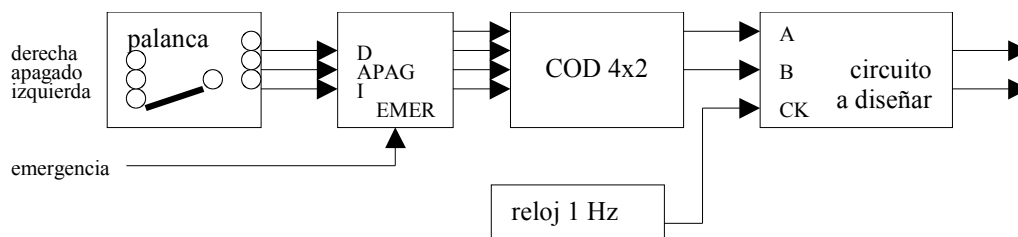


3) Se pretende diseñar el sistema de encendido de intermitencia de un coche. Para ello hay que diseñar un circuito secuencial que cumpla las siguientes especificaciones, de acuerdo con la figura adjunta.

Cuando la palanca se coloque en la posición DERECHA, se deberá encender y apagar de forma intermitente la luz identificada como D, de forma síncrona con un reloj de 1 Hz. Cuando la palanca se coloque en la posición IZQUIERDA, se deberá encender y apagar de forma intermitente la luz identificada como I, de forma síncrona con un reloj de 1 Hz. Cuando la palanca se coloque en la posición central (APAGADO) no se encenderá ninguna luz.

Cuando se active el interruptor de EMERGENCIA, se activarán ambas luces simultáneamente, y se desactivarán ambas de forma síncrona con el reloj, independientemente de la posición de la palanca, es decir, la entrada de emergencia tiene prioridad absoluta.

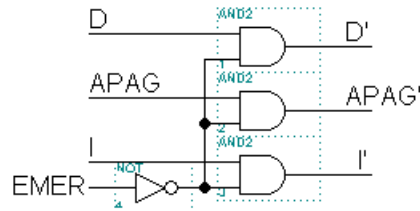
- Diseñar la circuitería adicional necesaria para que nunca entre más de una señal activa al codificador y poner una tabla de salida del codificador para todos los casos posibles.
- Definir los estados, su codificación y su diagrama.
- Realizar la tabla de excitación usando biestables D.
- Implementar el circuito.



NOTA: Aunque en un caso real sería imposible mecánicamente pasar la palanca de izquierda a derecha directamente, sin pasar por la posición Apagado, considerar que sí se puede al resolver este ejercicio.

En primer lugar debe diseñarse la circuitería necesaria para asegurar que nunca entra más de una señal activa en el codificador. Obsérvese en la figura del enunciado que las señales externas proceden de dos lugares diferentes. Por un lado, las señales D, APAG e I son generadas por un interruptor, por lo que por construcción es imposible que se activen dos simultáneamente. Por otro lado tenemos la señal EMER, que procede de la señal emergencia. Por lo tanto, lo que sí puede suceder es que ésta última y alguna de las anteriores se activen simultáneamente. Para evitarlo es necesario inhibir alguna de las dos cuando se dé este caso.

Puesto que la señal de emergencia es más prioritaria, lo que haremos será utilizarla como señal de ‘enable’ del resto, tal y como se muestra en la siguiente figura. Si EMER es 1, entonces EMER’ es 0 y las tres AND darán como resultado 0, independientemente de si también alguna de las señales I, D o APAG es 1. Si EMER es 0 entonces EMER’ es 1 y las puertas AND permitirán el paso de las 3 señales, de las cuales, como ya se ha comentado, sólo una estará activa.



Tal como se encuentran conectadas las entradas al codificador, el código de entrada en las señales A y B será el recogido por la siguiente tabla. Q1 intermitentes izquierdos y Q0 intermitentes derechos. Es un autómata de Moore, conocido el estado, se conoce la salida.

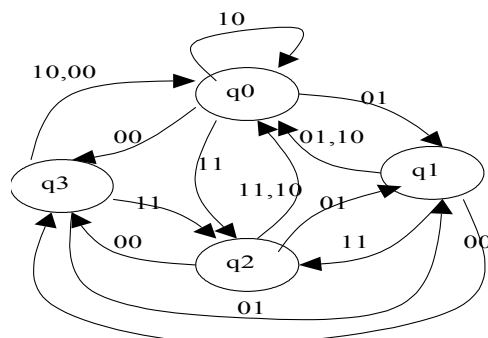
I A D E	Definición	Codificación Q1 Q0
0 0 0 1	Emergencia	0 0
0 0 1 0	Derecha encendida	0 1
0 1 0 0	Ambas apagadas	1 0
1 0 0 0	Izquierda encendida	1 1

1. Paso de las especificaciones verbales al diagrama de estados

Los cuatro posibles estados corresponden a las 4 combinaciones de los dos pilotos de intermitente. Hemos asignado la codificación tal y como se indica en la tabla bajo estas líneas, de manera que el mismo estado puede utilizarse como señal de salida para la activación de los intermitentes, simplificando de esta manera la función de salida.

Estado	Definición	Codificación
q ₀	Ambas apagadas	00
q ₁	Derecha encendida	01
q ₂	Izquierda encendida	10
q ₃	Ambas encendidas	11

Según esta codificación, el diagrama de estados del circuito secuencial a diseñar sería el siguiente.



Mientras nos encontremos en el estado q_0 , por ejemplo, y la entrada de A y B sea 10 el siguiente estado será q_0 , puesto que las luces deben permanecer apagadas. Supongamos que ahora se activa la señal I, solicitando un intermitente a la izquierda. En tal caso se recibirá el código 11 por A y B, por lo que, como se ve en el diagrama pasaremos al estado q_2 . Nótese como en el siguiente ciclo de reloj, si la entrada sigue siendo 11 el siguiente estado será q_0 , provocando que el piloto de la izquierda se apague. La razón de que se produzca esta transición, en lugar de permanecer en q_2 es que si no el piloto izquierdo permanecería iluminado permanentemente, sin parpadear. De esta manera, por tanto, mientras la entrada al circuito secuencial sea 11, la máquina de estados estará constantemente oscilando entre q_0 y q_2 , provocando así la intermitencia. El resto de situaciones funcionan de la misma manera.

2. Construcción de la tabla de estados

Tomando el diagrama de estados de la figura anterior debe generarse la tabla de estados, en la que a partir de las entradas A y B y el estado actual se obtendrá el siguiente estado, y en la que a partir de ambos, y con ayuda de la tabla de excitación, se calcularán las entradas a los biestables (D en este caso).

A	B	Q1(t)	Q0(t)		Q1(t+1)	Q0(t+1)		D1	D0
0	0	0	0		1	1		1	1
0	0	0	1		1	1		1	1
0	0	1	0		1	1		1	1
0	0	1	1		0	0		0	0
0	1	0	0		0	1		0	1
0	1	0	1		0	0		0	0
0	1	1	0		0	1		0	1
0	1	1	1		0	1		0	1
1	0	0	0		0	0		0	0
1	0	0	1		0	0		0	0
1	0	1	0		0	0		0	0
1	0	1	1		0	0		0	0
1	1	0	0		1	0		1	0
1	1	0	1		1	0		1	0
1	1	1	0		0	0		0	0
1	1	1	1		1	0		1	0

Minimización de las funciones e implementación del circuito

Simplificando por Karnaugh obtendremos las funciones de entrada a los biestables (D1 y D0) para el cálculo del nuevo estado. La función de salida (S1S0), como se ha comentado, consiste simplemente en conectar cada salida de los biestables a la señal de activación de cada una de los intermitentes.

$$\begin{aligned}
 D1 &= (A + B)' (Q1 Q0)' + AB (Q1' + Q0) & S1 &= Q1 \text{ (IZQUIERDOS)} \\
 D0 &= A' [B' (Q1 Q0)' + B (Q0' + Q1)] & S0 &= Q0 \text{ (DERECHOS)}
 \end{aligned}$$

4) Se desea diseñar el circuito de control de un semáforo de peatones, cuyo funcionamiento se describe a continuación.

Mientras no se active el pulsador ($P = 0$), el semáforo permanecerá por tiempo indefinido en VERDE. Cuando se pulse P, se encenderá en el siguiente ciclo de reloj la luz AMBAR, sin apagarse la VERDE, y transcurridos 5 segundos, se apagarán ambas y se encenderá la ROJA durante 40 segundos, finalizados los cuales se volverá a la situación inicial con sólo la luz VERDE encendida. Se supone que P se pulsa sólo cuando esté encendida la luz